DO NOW — RECALL

# Compilers, interpreters and assemblers

Name: _____ Class: _____ Date: _____

**1. What does LMC LDA do?**

**2. What is an executable?**

**3. What is a mnemonic?**

**4. Sorted [2,5,8,11,14,17,20,23,26], target23. Which midpoint indexes?**

DO NOW — CHECK / OBJECTIVES

# Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

**1. Model answer**

Loads value into ACC from mailbox.

**2. Model answer**

Machine-executable program representation.

**3. Model answer**

Human-readable instruction abbreviation.

**4. Model answer**

mid4→14, mid6→20, mid7→23; indexes 4,6,7.

**Vocabulary — know these terms**

compiler — Translates high-level source into target code.

interpreter — Runs a program through an execution runtime.

assembler — Translates assembly to machine encoding.

object code — Translator-generated machine-level representation.

# WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

### Guided 1 — original CHECK

Model answer

Explain the reasoning and justify the output.

IMPROVE: Explain one correction to your own reasoning.





NEW LEARNING

# Read, annotate and explain

### Understand

A compiler translates source code into another representation, often creating executable machine code via linking.

### Apply

An interpreter executes program statements with a runtime, often without a separate standalone compiled executable.

### Evaluate

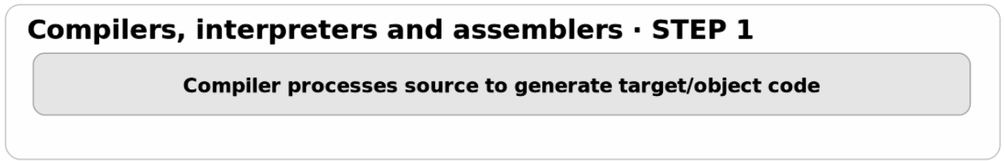
An assembler translates assembly mnemonics and operands into processor machine instructions; translation strategies can overlap in modern environments.

**Explain one step in your own words, using the correct technical terms.**

# New Learning · original lesson diagrams

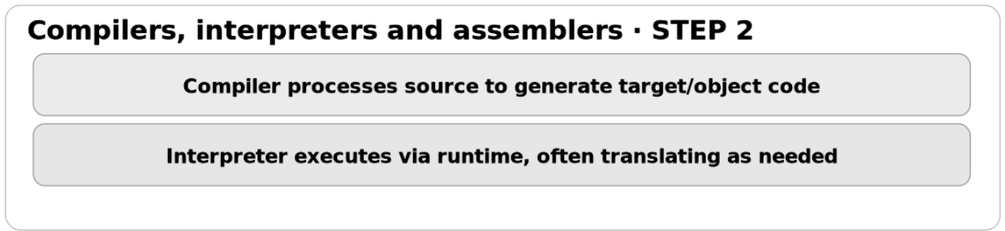
Diagrams appear after the related explanation. Annotate the original source visual.

### New Learning



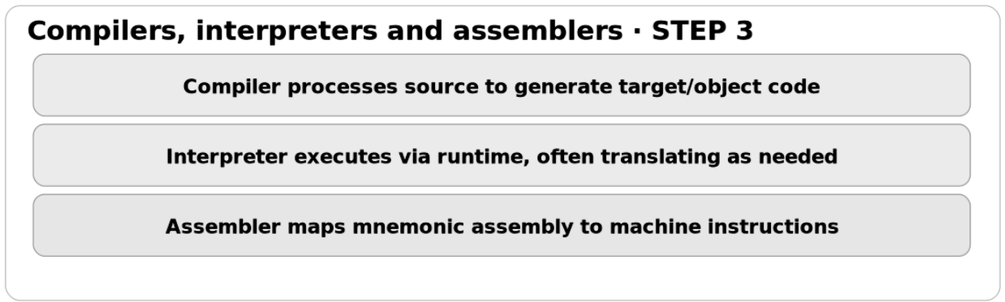

Compilers, interpreters and assemblers · STEP 1

### New Learning




Compilers, interpreters and assemblers · STEP 2

### New Learning




Compilers, interpreters and assemblers · STEP 3



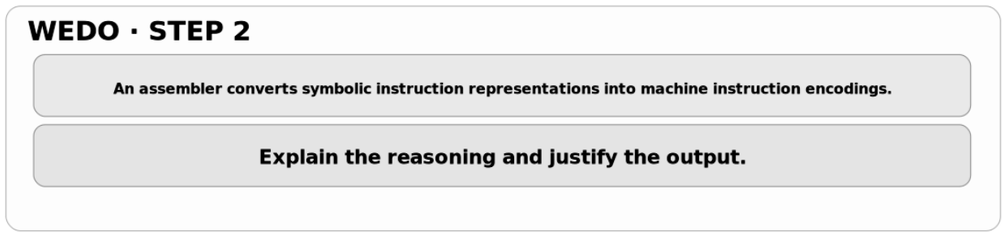


# We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

### A developer writes LMC-style mnemonics and wants binary instruction codes. Which translator is relevant?

An assembler converts symbolic instruction representations into machine instruction encodings.




WEDO · STEP 2

EXIT TICKET / REFLECTION

# Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

**An assembler translates assembly-language mnemonics into machine instructions.**

**How confident are you? Circle one: Confident / Getting there / Need help**

Explain your choice, identifying one concept you can now use independently.

**GREEN PEN — Correct and explain**

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.





I DO — WORKED EXAMPLE

# Follow the worked method

**A developer wants to distribute a compiled native executable. Which translator and what does it produce?**

**Predict first**

A compiler translates the original language into target/object code

**Model answer**

linking resolves required components.

# I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**A developer wants to distribute a compiled native executable. Which translator and what does it produce?**

A compiler translates the original language into target/object code

**IDO · STEP 1**

**A compiler translates the original language into target/object code**

IDO · STEP 1

**A developer wants to distribute a compiled native executable. Which translator and what does it produce?**

A compiler translates the original language into target/object code

**IDO · STEP 2**

**A compiler translates the original language into target/object code**

**linking resolves required components.**

IDO · STEP 2





**YOU DO — INDEPENDENT PRACTICE**

# Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

**Question 3**

Evaluate using an interpreter in a classroom programming exercise versus distributing a native compiled application. [6]

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

**Question 1**

Identify compiler, interpreter and assembler by typical role. [3]

**Question 2**

Explain two differences between compiling native code and interpreting source at runtime. [6]





WE DO — GUIDED PRACTICE

# Try → Discuss → Improve

# We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**A developer writes LMC-style mnemonics and wants binary instruction codes. Which translator is relevant?**

An assembler converts symbolic instruction representations into machine instruction encodings.




WEDO · STEP 1





# WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

**Guided 1: Predict first An assembler converts symbolic instruction representations into machine instruction encodings.**

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.